

1. Зачем нужны soft assertions

Обычный assertion бросает ошибку сразу. Если первое поле неверно, остальные проверки не выполняются.

Soft assertion запоминает каждое падение, продолжает блок, а в конце формирует одну ошибку со всеми найденными расхождениями.

Hard
ошибка №1 → стоп

→

Soft
№1 + №2 + №3 → отчёт

Когда применять: несколько независимых полей одного DTO, формы или API-ответа. Если без первой проверки продолжать опасно, оставьте её hard assertion.

2. Явная форма: видны все детали

```
SoftAssertions softly = new SoftAssertions();

softly.assertThat(product.name())
    .as("name")
    .isEqualTo("Keyboard");
softly.assertThat(product.available())
    .as("available")
    .isTrue();

softly.assertAll(); // бросает общую ошибку
```

1. Создали накопитель SoftAssertions.
2. Каждый softly.assertThat записывает ошибку в него.
3. assertAll() проверяет накопленное и завершает тест.

3. Лямбда и переменная softly

```
SoftAssertions.assertSoftly(softly -> {
    softly.assertThat(product.name()).isEqualTo("Keyboard");
    softly.assertThat(product.available()).isTrue();
});
```

assertSoftly ожидает функцию вида Consumer<SoftAssertions>. Библиотека сама создаёт объект SoftAssertions и передаёт его в вашу лямбду.

```
// Мысленная модель работы assertSoftly:
SoftAssertions object = new SoftAssertions();
consumer.accept(object); // object попадает в softly
object.assertAll();
```

softly — обычное имя параметра. Можно написать s или checks, но softly понятнее. Тип IDE выводит автоматически: SoftAssertions.

4. Почему product.name()

```
record Product(String name, BigDecimal price) { }

product.name(); // accessor record
product.price(); // accessor record
```

Компонент record хранится в private final поле, а компилятор создаёт публичный accessor с его именем. Рабочий контракт — name(). В этом вложенном учебном record внешний класс технически может увидеть private-поле как nestmate, но это особый случай: в отдельной модели product.name недоступен.

Тип модели	Обычно читаем поле
record Product(String name)	product.name()
Обычный JavaBean / Lombok @Data	product.getName()
Публичное поле	product.name, но так проектируют редко

5. Почему isEqualByComparingTo

```
BigDecimal a = new BigDecimal("99.90"); // scale 2
BigDecimal b = new BigDecimal("99.900"); // scale 3

a.equals(b); // false: значение + scale
a.compareTo(b); // 0: численно равны
```

AssertJ	Что проверяет
isEqualTo("99.90")	Для BigDecimal учитывает числовое значение и scale.
isEqualByComparingTo("99.90")	Использует сравнение по числовому значению; 99.900 равно 99.90.

Для денег чаще важна сумма, поэтому подходит isEqualByComparingTo. Если формат/scale входит в контракт — используйте isEqualTo.

6. Главная ловушка

```
SoftAssertions softly = new SoftAssertions();
softly.assertThat(actual).isEqualTo(expected);
// забыли softly.assertAll() → тест может стать ложно зелёным
```

Форма SoftAssertions.assertSoftly(...) безопаснее: она вызывает финальную проверку автоматически, даже если вы её не написали.

7. Рецепт на каждый день

1. Выберите один объект и связанные свойства.
2. Откройте SoftAssertions.assertSoftly(softly -> { ... }).
3. Для каждого свойства напишите отдельный softly.assertThat(actualField).
4. Добавьте as("поле/смысл") до assertion.
5. Используйте типовой assertion: строка, число, список, BigDecimal.
6. Закройте блок; assertAll выполнится автоматически.

Мантра: softly не является данными продукта. Это накопитель проверок, созданный AssertJ и переданный в лямбду.